

Project Review — quality assessment & suggestions

An honest engineering review of AssetDesk as of 2026-07-06 (app v6.12.0). Written to be actionable: each finding says what, why it matters, and what to do.

Overall verdict

Good — notably above average for a Forge app of this size, with a few structural gaps. The backend is genuinely well-engineered: the async job architecture is correct and consistently applied, the security model (AQL-as-boundary + server-side ownership re-verification) is sound, platform limits (25 s invoke ceiling, 128 KB KVS cap, 1,000-object AQL cap, rate limits) are not just respected but documented in-code at the exact point they bite. The in-code commentary is exceptional — most functions explain *why*, often with the production bug that motivated them.

The main weaknesses are the absence of any automated testing, two very large frontend files, no version control, and a handful of hardening gaps listed below.

Scorecard

Dimension	Score	One-line justification
Architecture	★★★★★	Async-job pattern, caching, and bounded concurrency are all correct and consistently applied
Security	★★★★☆	Strong core (ownership verification, AQL escaping); a few edges to close (below)
Code quality	★★★★☆	Clear naming, superb comments, consistent patterns; two 2,300+-line files drag it down
Reliability	★★★★☆	Defensive error handling everywhere; no retries/DLQ on queue jobs yet
Testing	★☆☆☆☆	None. All verification is manual against the dev site
Operability	★★★★☆	Good structured logging; no version control, no CI, single environment discipline
Documentation	★★★★★	CLAUDE.md/AGENTS.md + heavy in-code commentary + this docs set

Strengths worth preserving

- The async job template** (`startAssetLoadJob` → queue → consumer → KVS poll) is implemented three times *identically*, including the subtle pending-written-before-push ordering and identity-captured-in-sync-context rules. Copy it verbatim for any future heavy work.
- Security posture on writes.** `updateAssetAttribute` re-verifies ownership server-side with `asApp()` before every PUT; the CSV importer and export consumer never trust client-supplied data for anything security-relevant.

3. **Platform-limit awareness.** `escapeAqlValue` on every interpolation, `CACHE_MAX_ASSETS` and export chunking for the 128 KB cap, chunked concurrency for rate limits, explicit handling of the 1,000-object AQL cap with an admin-facing warning.
 4. **Failure honesty in UX.** Unverified $\neq$ deleted in drift detection; CSV warnings vs errors are distinct; `diagnoseCaller` turns "I see nothing" into a structured report. This is the kind of operational empathy that reduces support load.
 5. **Idempotent CSV re-runs** via unchanged-detection — re-importing the same file reports `unchanged` rather than blindly rewriting every row.
-

Suggestions (prioritized)

P0 — do these first

1. **Put the project under git.** There is no version control (`git init` has never been run). Every risk below is amplified by having no history, no diff review, and no rollback. One command plus a `.gitignore` (`node_modules/` , `.forge/`) fixes it. This also unlocks PR-based review and CI later.
2. **Add resolver-level authorization guards on admin resolvers.** All four UI modules share one resolver function, and `saveConfig` / `applyReconciliation` / `getAssetsForUser` etc. do not check who is calling — they rely on only the admin page invoking them. A portal customer's browser session *can* invoke any defined resolver name. Mitigations exist (`asUser()` calls fail without the caller's own Assets permission), but `saveConfig` writes straight to KVS with no product-API call to fail — an unlicensed caller invoking `saveConfig` could corrupt or clear the app's configuration. Fix: at the top of each admin resolver, reject unlicensed callers (`isUnlicensedCaller(context)`) and ideally verify Jira-admin permission via `/rest/api/3/mypermissions?permissions=ADMINISTER` .
3. **Cap `previewCsvImport` / consumer input size.** A very large attached CSV is read fully into memory (`contentRes.text()`) inside a 256 MB function and parsed row-by-row. Add a size check on the attachment metadata (already returned by `getIssueCsvAttachments`) and refuse files above a sane bound (e.g. 5 MB / ~20k rows) with a clear message, before download.

P1 — high value, moderate effort

4. **Introduce automated tests for the pure logic.** No framework exists today. The highest-value, zero-Forge-dependency targets are in `shared.js` : `buildAqlFromRow` / `buildAqlCandidates` / `buildFilterAql` (the security- and correctness-critical string builders), `normalizeDateValue` , `matchCsvHeadersToAttributes` , `parseCsvRows` , `escapeAqlValue` , `clampUserAssetLimit` . A plain `node --test` (built into Node 24) suite needs no new dependencies. Mock-heavy resolver tests can come later; the AQL builders alone would catch the class of bug that has historically cost the most debugging time here.
5. **Job-key hygiene: TTLs / cleanup.** `asset-load-job:*` , `csv-import-job:*` , and abandoned `export-job:*` (+ chunks, if the user closes the tab before the poll that deletes them) accumulate in KVS forever. Options: attach `expiresAt` and lazily delete on read; or a scheduled trigger

(`scheduledTrigger` module) sweeping keys older than a day. Low urgency at current volume, but unbounded growth in a keystore is a slow leak.

6. **Split the two mega-files.** `frontend/index.jsx` (2,300 lines) and `ConfigurePage.jsx` (2,400 lines) each hold 10+ components. Extracting obvious units (`EditAssetModal` + `AttributeField` , `FilterBar` , `ExportButtons` , the tables; `ColumnVisibilitySection` , the AQL row editor, the drift banner) into `src/frontend/components/` would shrink each file by more than half. Note manifest `resources` point at specific files — keep the entry files, move the pieces.
7. **Delete dead code.** `ObjectTypeColumns` in `ConfigurePage.jsx` (~line 512) is unused (superseded by `ObjectTypeColumnsControlled`) — confirmed no render call sites. It has already caused one near-miss edit ambiguity; remove it.
8. **Queue-consumer retry/dead-letter behavior.** Consumers currently swallow unexpected exceptions into a `status: 'error'` KVS write. Forge's Async Events support retries (`InvocationError` / `retry` options); today a transient failure (e.g. a 429 while downloading the CSV) permanently fails the job. Consider: rethrow-with-retry for transient classes, keep the KVS error write for permanent ones.

P2 — nice to have

9. **Config versioning safety on save.** `saveConfig` full-replaces the config. Two admins editing simultaneously last-write-wins silently. The `version` field already exists — have the frontend send the version it loaded and let `saveConfig` reject on mismatch ("config changed since you loaded it").
10. **CSV import for non-agents.** The `jira:issuePanel` module is invisible to unlicensed/portal users (an open question from earlier work). If customer- initiated imports are ever needed, the JSM-side module options (`jiraServiceManagement:portalRequestDetailPanel` etc.) are the place to look — same resolvers would work, but auth branching would need the unlicensed treatment (`asApp()` + `read:attachment` implications reviewed).
11. **Structured log levels.** Logging is thorough but all `console.log/warn/error` with ad-hoc prefixes (`[resolveAssetList]` , ...). A tiny `log(scope, level, msg)` helper would make `forge logs` filtering easier and keep prefixes consistent.
12. **README.md at repo root.** There's rich internal documentation (CLAUDE.md, AGENTS.md, now docs/) but no plain README for a human landing on the folder. Three paragraphs + a pointer to `docs/` suffices.
13. **Pin the Forge platform quirks list.** UI Kit limitations discovered the hard way (no `Tabs.onChange` , no `Table` , `viewportSize` behavior, host-page layout constraints that led to the "accept mild squeeze" table decision) are spread across code comments. `docs/frontend.md` §5 now starts this; keep adding to it when the platform surprises you — it's the cheapest knowledge to lose.

Known accepted trade-offs (documented, not defects)

These look like issues but are deliberate; don't "fix" them without revisiting the original reasoning:

- **Everything `asApp()` in queue consumers** — no user session exists there; ownership scoping comes from AQL, and this was an explicit, reviewed decision.
- **Hidden attributes are enforced per object type, not inherited from parent types.** A parent-type-inheritance model was attempted (2026-07-06) and **reverted** at the user's request after it didn't behave as expected in practice. Hiding an attribute on one type does not affect sibling/child types that share the same underlying attribute definition. If revisiting: the live API's attribute records do carry the true owner in `attr.objectType.id`, and attribute ids are shared across inheriting types — the concept is viable, but the UX/semantics need rethinking before another attempt.
- **The table accepts mild horizontal squeeze** rather than forcing width on the host Jira page — forcing it broke the portal's own layout grid.
- **Status filters are exact-match** while text filters are substring — status values come from a closed dropdown, and AQL `LIKE` doesn't work on status reference fields anyway.
- **`maxUserAssetLimit` truncation is by merge order**, not any user-meaningful ranking — acceptable because the banner tells the user truncation happened.